



پروژه شماره ۱

درس مبانی رمزنگاری

دانشگاه مهندسی و علوم کامپیوتر

مدرس: دکتر سالاری فرد

آذر ۱۴۰۳

ساختار رمزنگاری فایستلی یک روش رمزگذاری^۱ متقارن است که داده‌های متن ساده را به دو نیمه تقسیم می‌کند که معمولاً به آنها نیمه چپ و راست می‌گویند. در هر دور، تنها یک نیمه با استفاده از یک تابع دور و یک زیرکلید پردازش می‌شود، سپس خروجی با نیمه دیگر XOR می‌شود، که سپس در دور بعدی دو نیمه با هم جابجا می‌شوند. ویژگی مهم این ساختار خاصیت خود وارونگی آن می‌باشد، به این معنا که نیازی به پیاده‌سازی مازول رمزگشایی وجود ندارد و می‌توان از همان مازول رمزگذاری به منظور رمزگشایی استفاده کرد. ساختارهای فایستلی به دلیل تعادل بین سادگی و امنیت، پایه ای برای بسیاری از رمزهای بلوکی از جمله DES هستند. طراحی آن‌ها اجازه می‌دهد تا فرآیندهای رمزگذاری و رمزگشایی^۲ مشابه باشند و پیچیدگی اجرا را کاهش دهند.

^۱ Encryption

^۲ Decryption



فهرست

۳	۱ صورت پروژه
۴	۲ ساختار فیستلی
۵	۳ رمز COCO-128
۵	۳-۱ جعبه های جایگزینی
۷	۳-۲ مازول مدیریت کلید
۸	۳-۲-۱ مازول های pre-key-1 و pre-key-2
۹	۳-۲-۲ مازول های key-gen-i
۱۰	۳-۳ تابع دور
۱۱	۴ پیاده سازی طرح رمز COCO-128 در مُد EBC
۱۲	۵ پیاده سازی مُد OFB، CTR و CBC
۱۲	۵-۱ مُد OFB
۱۲	۵-۲ مُد CTR
۱۳	۵-۳ مُد CBC
۱۵	۶ پیوست



۱ صورت پروژه

در این پروژه از شما خواسته شده است، یک رمز قالبی مبتنی بر ساختار فیستلی را پیاده‌سازی کنید. در این راستا در گام اول موارد ذیل باید پیاده‌سازی شود:

(۱) ماژول مدیریت جعبه‌های جایگذاری

(۲) ماژول مدیریت کلید

(۳) تابع دور

(۴) ساختار فیستلی مبتنی بر تابع دور

در نتیجه پیاده‌سازی موارد فوق، به یک رمز قالبی دست پیدا کرده‌ایم که می‌تواند یک بلوک π -بیتی را رمزگذاری کند و π -بیت متن رمز شده را در خروجی قرار دهد. در گام دوم باید مدهای ذیل برای رمز قالبی مذکور را پیاده‌سازی کنیم:

(۱) مُد EBC

(۲) مُد OFB

(۳) مُد CTR

(۴) مُد CBC

حال در گام آخر برای صحت سنجی رمز پیاده‌سازی شده لازم است، یک متن را پس از کدگذاری مناسب با استفاده از مدهای مختلف رمزگذاری و سپس رمزگشایی کنید.

(!) توجه: نحوه‌ی کدگذاری متن به خود شما واگذار شده است.

- سوالات و ابهامات خود را ترجیحاً در گروه Discussion درس مطرح کنید یا به اعضای تیم حل تمرین پیام دهید.
- در صورت مشاهده هر گونه تقلب نمره صفر برای طرفین در نظر گرفته خواهد شد.
- زبان‌های مجاز برای پیاده‌سازی این پروژه، جاوا و پایتون می‌باشد.
- نوشتن گزارش الزامی می‌باشد و ۱۰ درصد از ارزشیابی پروژه توسط گزارش انجام می‌شود.
- فرمت نام‌گذاری پروژه به صورت زیر باشد.

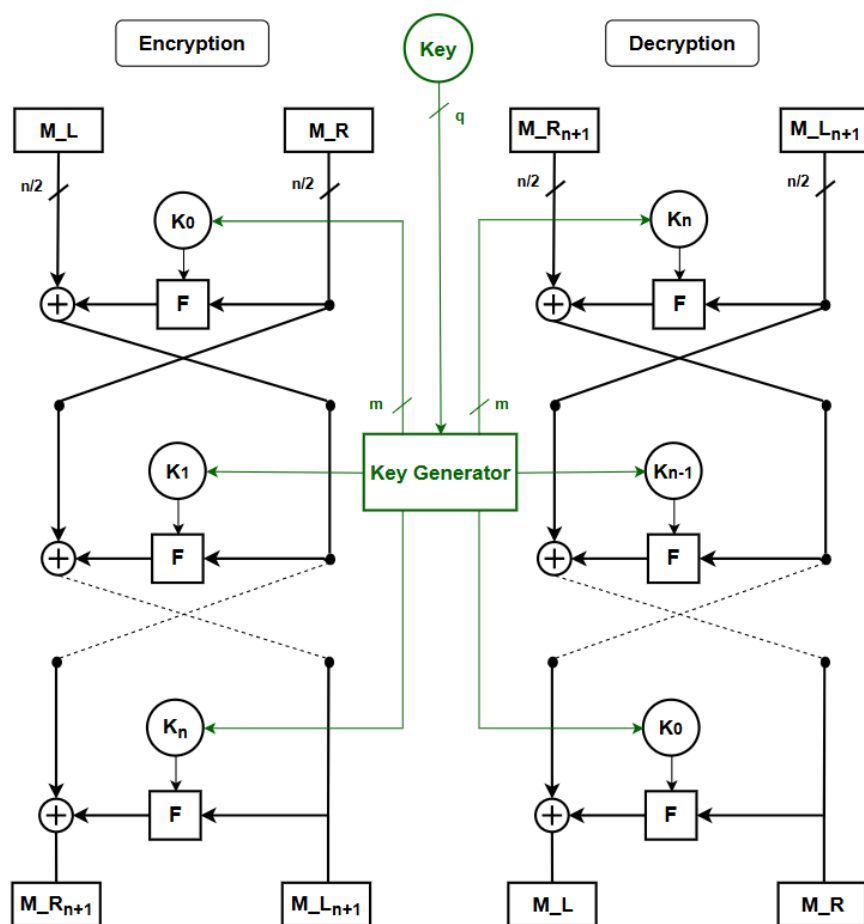
PROJ-1_[Student-ID]_[Student_Name]

موفق باشید.



۲ ساختار فیستلی^۱

فرآیند رمزگذاری رمزی فیستلی شامل دوره‌های متعددی از پردازش متن ساده است. هر دور شامل مرحله جایگذاری و سپس مرحله جایگشت است.



شکل ۱ رمزگذاری و رمزگشایی ساختار فیستلی

ساختار رمزنگاری فیستلی به گونه‌ای طراحی شده است که رمزگشایی را مانند رمزگذاری با استفاده از یک فرآیند برگشت پذیر آسان کند. مزیت اصلی ساختار فیستلی این است که رمزگشایی را با معکوس کردن ترتیب زیرکلیدها و اعمال همان فرآیند به صورت معکوس امکان پذیر می‌کند. متن ساده بدون نیاز به الگوریتم رمزگشایی جداگانه!!!

در شکل ۱ ساختار رمزگذاری و رمزگشایی فیستلی نشان داده شده است. با کمی دقت بیشتر به این ساختار می‌توان به ویژگی‌هایی پی‌برد، که رمزهای گوناگون مبتنی بر ساختار فیستلی را از یکدیگر متمایز می‌کنند. این ویژگی‌های عبارتند از:

¹ Feistel Structure



- (۱) تابع دور^۱
- (۲) ماژول مدیریت کلید^۲
- (۳) تعداد دور
- (۴) طول قطعه
- (۵) طول کلید و زیرکلید

تمامی موارد فوق باید در پیاده‌سازی یک رمز مبتنی بر ساختار فیستلی ملاحظه شوند.

۳ رمز COCO-128

رمز COCO-128، یک رمز بلوکی مبتنی بر ساختار فیستلی می‌باشد (شکل ۱). مشخصات این رمز در مقایسه با طرح رمز DES در جدول ۱ آورده شده است.

نام رمز	تعداد دور (بیت)	طول قطعه (بیت)	طول کلید (بیت)	طول زیرکلید (بیت)
COCO-128	۱۶	۱۲۸	۱۲۸	۶۴
DES	۱۶	۶۴	۶۴	۴۸

جدول ۱ مشخصات رمز COCO-128 در مقایسه با رمز DES

۳-۱ جعبه‌های جایگزینی^۳

در این طرح رمز از ۸ جعبه جایگزینی در دو نوع مختلف استفاده شده است. این جداول شامل ۸ ستون و ۳۲ سطر می‌باشند. بنابراین جعبه‌ها یک ورودی ۸-بیتی دریافت می‌کنند؛ ۵-بیت اول از سمت چپ را برای تعیین سطر و ۳-بیت باقی‌مانده را برای تعیین ستون استفاده می‌کنند.

نوع اول این جعبه‌ها در تابع دور این طرح رمز استفاده می‌شود. جدول ۲، اولین جعبه جایگزینی نوع اول را نشان می‌دهد. این جداول ۸-بیت ورودی را به ۸-بیت خروجی نگاشت می‌کنند. برای روشن شدن بیشتر موضوع مثال زیر را در نظر بگیرید:

$$\text{input} = (123)_{10} = (01111011)_2 \rightarrow \text{Sbox1}(123) = 168 \text{ [row} = (01111)_2 = 15, \text{column} = (011)_2 = 3]$$

(!) توجه: اعداد موجود در جداول نوع اول همگی دسیمال و ۸-بیتی هستند.

سه جعبه جایگزینی باقی‌مانده این نوع نیز بر اساس جعبه اول به دست می‌آیند:

$$\begin{aligned} \text{Sbox2}[x] &= \text{Sbox1}[x] \lll 1; \\ \text{Sbox3}[x] &= \text{Sbox1}[x] \lll 7; \\ \text{Sbox4}[x] &= \text{Sbox1}[x] \lll 1; \end{aligned}$$

¹ Round Function

² Key schedule

³ Substitution Box



عملگر "<<<" یک شیفت چرخشی^۱ می‌باشد. به مثال زیر توجه کنید:

$$130 \lll 1 = 5 : (130)_{10} = (10000010) \rightarrow (00000101)_{10} = (5)_{10}$$

Sbox1								
	0	1	2	3	4	5	6	7
00	112	130	44	236	179	39	192	229
01	228	133	87	53	234	12	174	65
02	35	239	107	147	69	25	165	33
03	237	14	79	78	29	101	146	189
04	134	184	175	143	124	235	31	206
05	62	48	220	95	94	197	11	26
06	166	225	57	202	213	71	93	61
07	217	1	90	214	81	81	86	77
08	139	13	154	102	251	204	176	45
09	116	18	43	32	240	177	132	153
A	223	76	203	194	52	126	118	5
B	109	183	169	49	209	23	4	215
C	20	88	58	97	222	27	17	28
D	50	15	156	22	83	24	242	34
E	254	68	207	178	195	181	122	145
F	36	8	232	168	96	252	105	80
11	170	208	160	125	161	137	98	151
12	84	91	30	149	224	255	100	210
13	16	196	0	72	163	247	117	219
14	138	3	230	128	9	63	221	148
15	135	92	131	2	205	74	144	51
16	115	103	246	243	157	127	191	226
17	82	155	216	38	200	55	198	59
18	129	150	111	75	19	190	99	46
19	233	121	167	140	159	110	188	142
1A	41	245	249	182	47	253	180	89
1B	120	152	6	106	231	70	113	186
1C	212	37	171	66	136	162	141	250
1D	114	7	185	85	248	238	172	10
1E	54	73	42	104	60	56	241	164
1F	64	40	211	123	187	201	67	193
20	21	227	173	244	119	199	128	158

جدول ۲ Sbox1 از نوع اول

¹ Rotational Shift

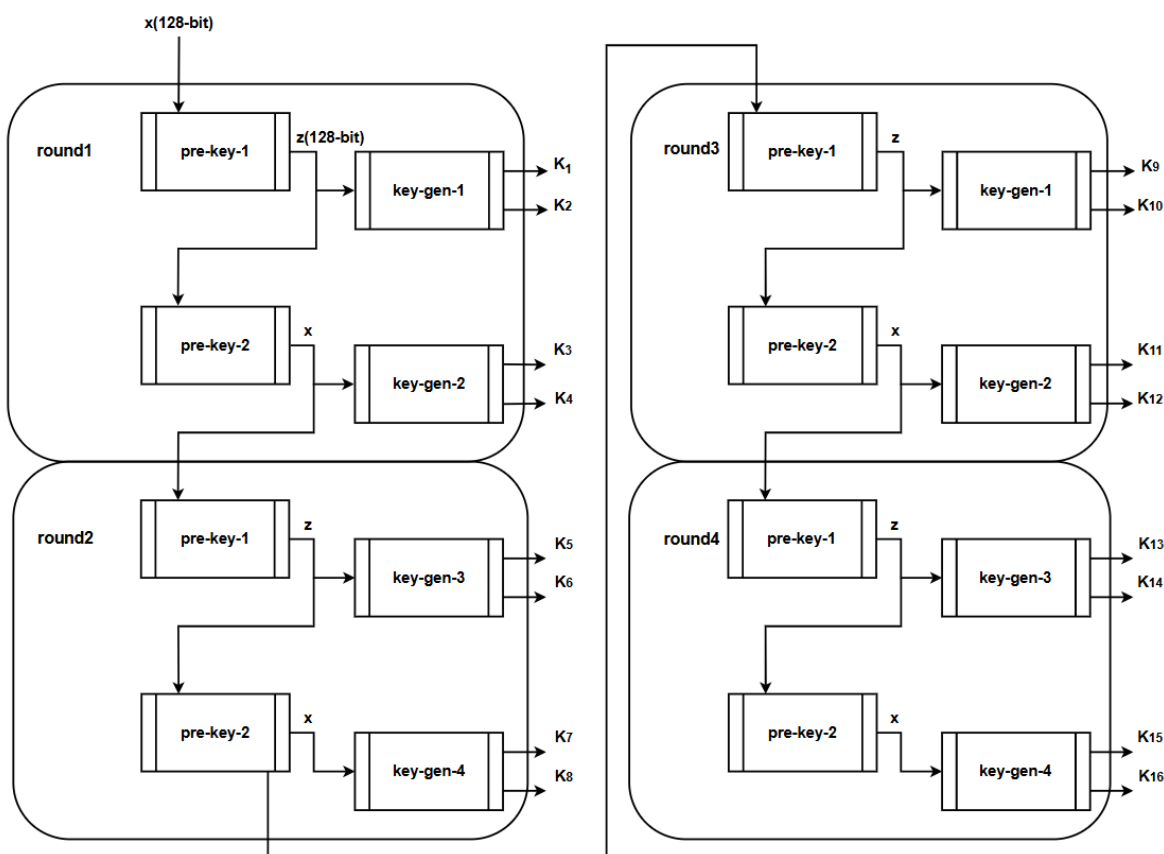


نوع دوم این جعبه‌ها در ماژول مدیریت کلید استفاده می‌شوند. این جداول ۸-بیت ورودی را به ۳۲-بیت خروجی نگاشت می‌کنند. «می‌توانید این جداول را در فایل‌های txt پیوست شده؛ مشاهده کنید.»

(۱) توجه: اعداد موجود در جداول نوع دوم همگی هگزا دسیمال و ۳۲-بیتی هستند.

۲-۳ ماژول مدیریت کلید

همانطور که در بخش پیشین گفته شد؛ این طرح رمز دارای یک کلید اصلی ۱۲۸-بیتی و زیرکلیدهای ۶۴-بیتی می‌باشد. از آنجایی که این رمز در ۱۶ دور از ساختار فیستلی عمل می‌کند؛ بنابراین ماژول مدیریت کلید در خروجی خود ۱۶ زیرکلید ۶۴-بیتی تولید خواهد کرد. در شکل ۲ شمای کلی ماژول مدیریت کلید نشان داده شده است. مطابق آنچه در این شکل نشان داده شده است؛ ماژول مدیریت کلید در ۴ دور اجرا می‌شود. در هر دور آن ۸ زیرکلید ۶۴-بیتی تولید می‌شود. هر دور شامل ۴ ماژول کاملاً مبتنی بر جعبه‌های جایگزینی است. ابتدا متغیر Z را به منظور ذخیره موقت تغییرات اعمال شده بر روی کلید اصلی (X) در هر دور تعریف کنید. کلید X در دور اول وارد ماژول $pre-key-1$ می‌شود؛ تغییرات اعمال شده بر روی آن در متغیر Z ذخیره شده و به ماژول $key-gen-1$ ارسال می‌شود تا دو زیرکلید اول تولید شود. همچنین داده‌های موجود در متغیر Z به ماژول $pre-key-2$ نیز ارسال می‌شوند. در این ماژول دوباره تغییراتی بر روی Z اعمال می‌شود و به ماژول $key-gen-2$ ارسال شده تا دو زیرکلید دیگر تولید شود. مطابق شکل این فرآیند ۴ دور تکرار خواهد شد تا به ۱۶ زیرکلید ۶۴-بیتی دست یابیم.



شکل ۲ شمای کلی ماژول مدیریت کلید



۱-۲-۳ مازول‌های pre-key-1 و pre-key-2

در شکل ۳ می‌توان جزئیات بیشتری از مازول pre-key-2 را مشاهده کرد. به منظور درک ساختار این مازول مثال زیر را در نظر بگیرید که در دور اول هستیم.

ورودی ۱۲۸-بیتی زیر را در نظر بگیرید:

$x: \text{\textbackslash x04\textbackslash xf1\textbackslash x91\textbackslash x30\textbackslash x96\textbackslash x98\textbackslash x02\textbackslash x65\textbackslash x63\textbackslash x03\textbackslash x75\textbackslash x25\textbackslash x46\textbackslash x12\textbackslash x19\textbackslash x57}$

ابتدا ورودی را به چهار قسمت تقسیم می‌کنیم:

$x[0:4]$	$= \text{\textbackslash x04\textbackslash xf1\textbackslash x91\textbackslash x30}$
$x[4:8]$	$= \text{\textbackslash x96\textbackslash x98\textbackslash x02\textbackslash x65}$
$x[8:12]$	$= \text{\textbackslash x63\textbackslash x03\textbackslash x75\textbackslash x25}$
$x[12:16]$	$= \text{\textbackslash x46\textbackslash x12\textbackslash x19\textbackslash x57}$

حال مطابق شکل ۳ عمل می‌کنیم؛ $x[0:4]$ را با خروجی $Sbox5(x[13])$ ، $Sbox6(x[15])$ ، $Sbox7(x[12])$ ، $Sbox8(x[14])$ و $Sbox7(x[8])$ با پیمانه ۲ جمع کرده (عملیات XOR) و در $z[0:4]$ ذخیره می‌کنیم.

$z[0:4] = x[0:4] \wedge Sbox5(x[13]) \wedge Sbox6(x[15]) \wedge Sbox7(x[12]) \wedge Sbox8(x[14]) \wedge Sbox7(x[8])$

$x[8:12]$ را با خروجی $Sbox5(z[0])$ ، $Sbox6(z[2])$ ، $Sbox7(z[1])$ ، $Sbox8(z[3])$ و $Sbox8(x[10])$ با پیمانه ۲ جمع کرده و در $z[4:8]$ ذخیره می‌کنیم.

$z[4:8] = x[8:12] \wedge Sbox5(z[0]) \wedge Sbox6(z[2]) \wedge Sbox7(z[1]) \wedge Sbox8(z[3]) \wedge Sbox8(x[10])$

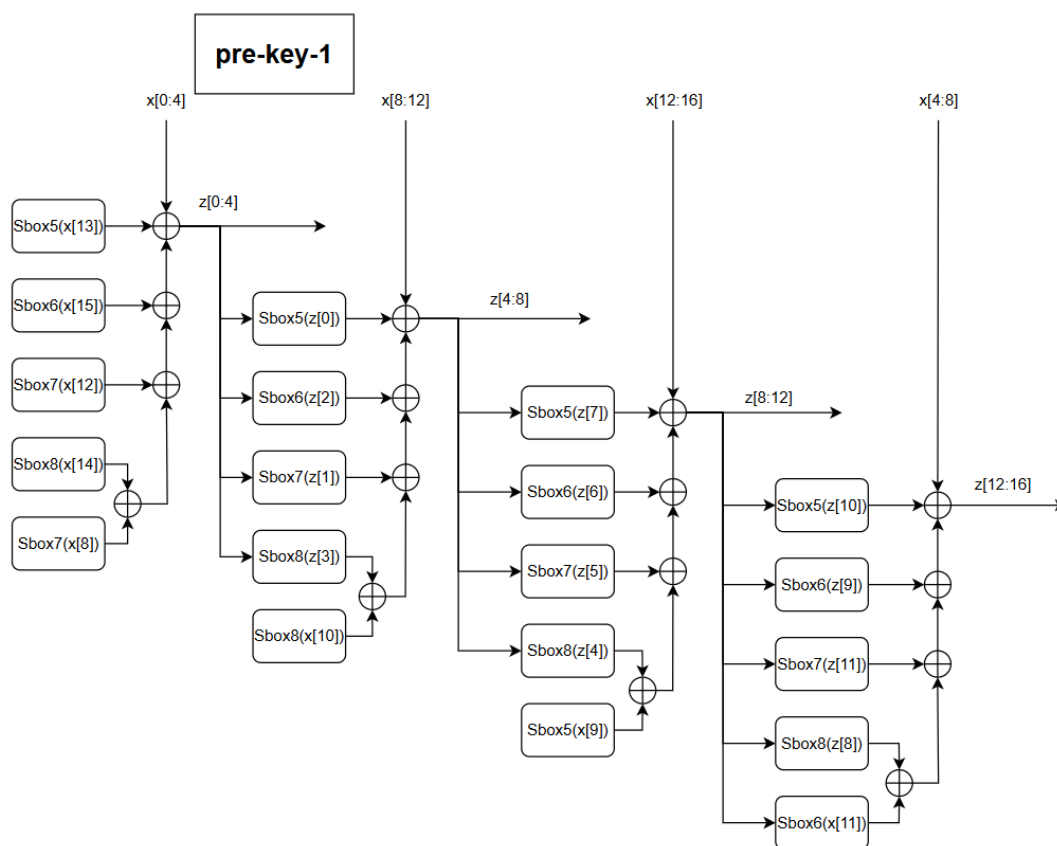
$x[12:16]$ را با خروجی $Sbox5(z[7])$ ، $Sbox6(z[6])$ ، $Sbox7(z[5])$ ، $Sbox8(z[4])$ و $Sbox5(x[9])$ با پیمانه ۲ جمع کرده و در $z[8:12]$ ذخیره می‌کنیم.

$z[8:12] = x[12:16] \wedge Sbox5(z[7]) \wedge Sbox6(z[6]) \wedge Sbox7(z[5]) \wedge Sbox8(z[4]) \wedge Sbox5(x[9])$

$x[4:8]$ را با خروجی $Sbox5(z[10])$ ، $Sbox6(z[9])$ ، $Sbox7(z[11])$ ، $Sbox8(z[8])$ و $Sbox6(x[11])$ با پیمانه ۲ جمع کرده و در $z[12:16]$ ذخیره می‌کنیم.

$z[8:12] = x[4:8] \wedge Sbox5(z[10]) \wedge Sbox6(z[9]) \wedge Sbox7(z[11]) \wedge Sbox8(z[8]) \wedge Sbox6(x[11])$

در گام بعد $z[0:16]$ که آماده شده است را به دو مازول per-key-2 و key-gen-1 ارسال می‌کنیم. ساختار per-key-2 در پیوست آمده است. عملکرد این مازول نیز همانند مازول per-key-1 در مثال فوق است با این تفاوت مقدار ورودی آن z می‌باشد و مقادیر تغییر یافته را بر روی x می‌ریزد.



شکل ۳ ساختار ماژول pre-key-1

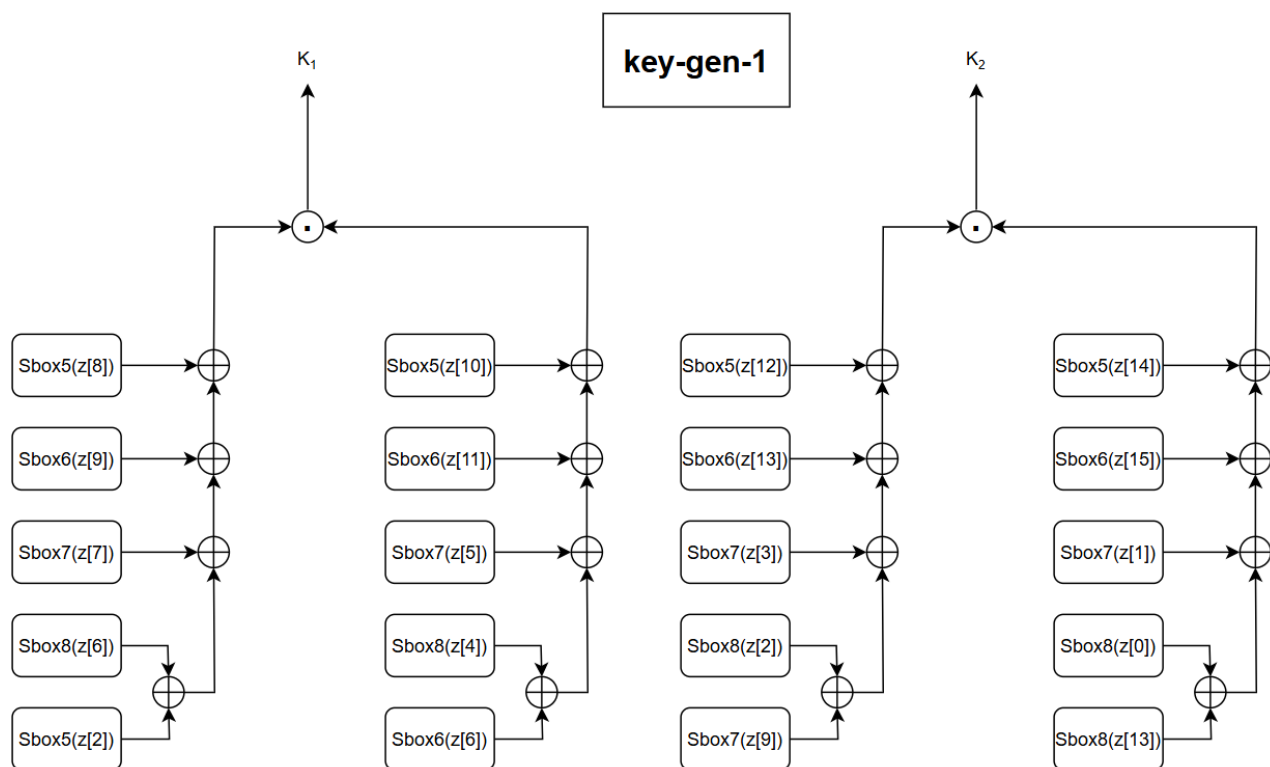
۲-۲-۳ ماژول های key-gen-i

ماژول های key-gen-i ورودی خود را از یکی از دو ماژول per-key-1 یا per-key-2 دریافت می کنند و با کمک جعبه های جایگزینی مربوط به ماژول مدیریت کلید اقدام به تولید زیر کلیدهای ۶۴-بیتی خواهند کرد. شکل ۴ ساختار ماژول key-gen-1 را نشان می دهد؛ به منظور درک بهتر ساختار این ماژول ها مثال زیر را در نظر بگیرید.

فرض کنید که در دور اول هستیم مقدار z را از pre-key-1 دریافت کرده ایم:

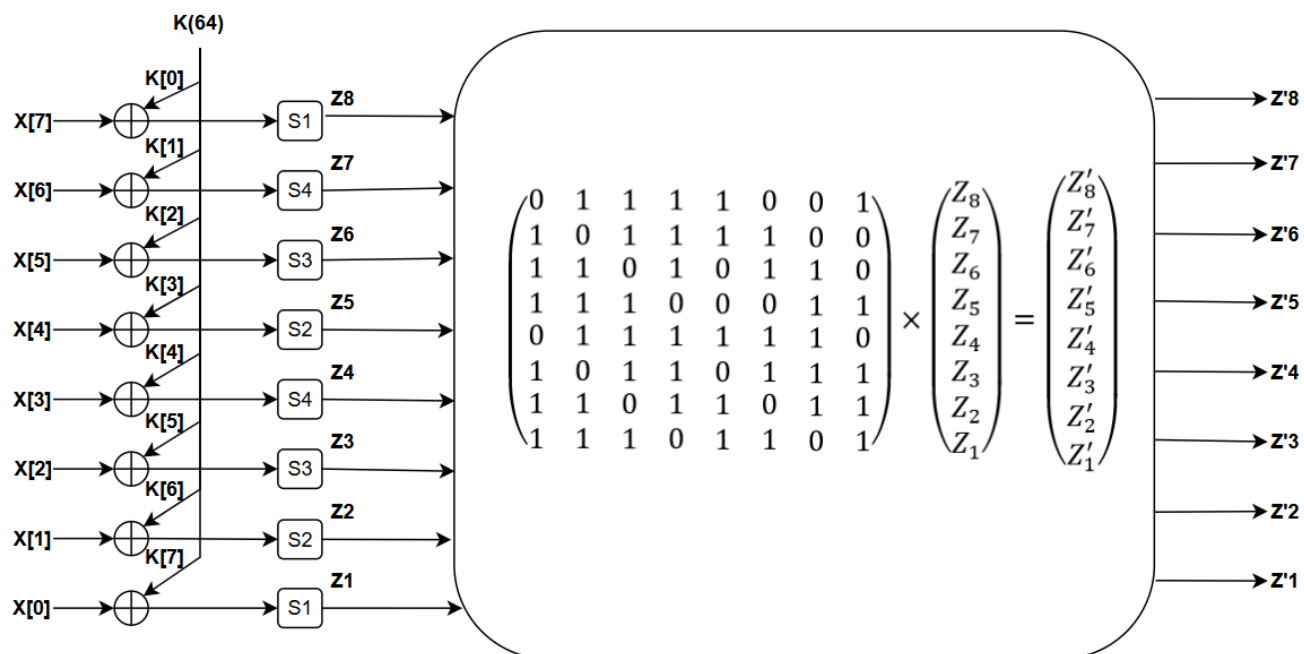
$$\begin{aligned} K_a &= \text{Sbox5}(z[8]) \wedge \text{Sbox6}(z[9]) \wedge \text{Sbox7}(z[7]) \wedge \text{Sbox8}(z[6]) \wedge \text{Sbox5}(z[2]) \\ K_b &= \text{Sbox5}(z[10]) \wedge \text{Sbox6}(z[11]) \wedge \text{Sbox7}(z[5]) \wedge \text{Sbox8}(z[4]) \wedge \text{Sbox6}(z[6]) \\ K1 &= K_a \mid K_b \\ K_c &= \text{Sbox5}(z[12]) \wedge \text{Sbox6}(z[13]) \wedge \text{Sbox7}(z[3]) \wedge \text{Sbox8}(z[2]) \wedge \text{Sbox7}(z[9]) \\ K_d &= \text{Sbox5}(z[14]) \wedge \text{Sbox6}(z[15]) \wedge \text{Sbox7}(z[1]) \wedge \text{Sbox8}(z[0]) \wedge \text{Sbox8}(z[13]) \\ K2 &= K_c \mid K_d \end{aligned}$$

ساختار سایر ماژول ها را می توانید در [پیوست](#) مشاهده کنید.



شکل ۴ ساختار ماژول key-gen-1

۳-۳ تابع دور



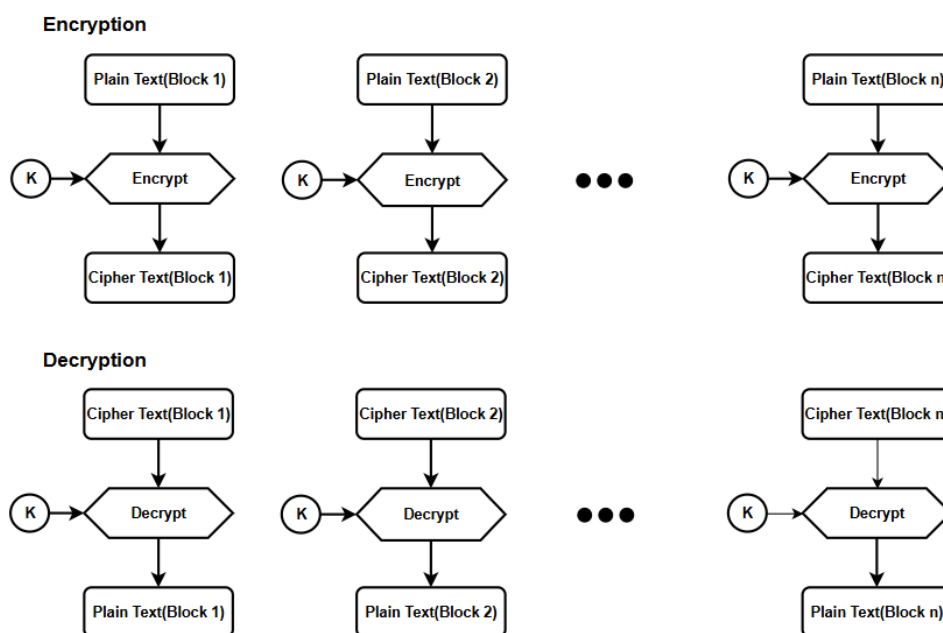
شکل ۵ ساختار تابع دور رمز COCO-128



شکل ۵ ساختار تابع دور را در رمز COCO-128 نشان می‌دهد. مطابق شکل فوق این تابع دو ورودی دارد، یک زیرکلید ۶۴-بیتی و یک قطعه ۶۴-بیتی از متن معلوم. ابتدا هر بایت از ورودی‌ها را با یک دیگر در مبنای ۲ جمع می‌کند (عملیات XOR). سپس نتایج آن‌ها را به جعبه‌های جایگزینی مختص تابع دور ارسال می‌کند. همانطور که در بخش قبل گفته شد خروجی این جعبه‌ها یک بایت می‌باشند. در نهایت به منظور اعمال جایگشت مناسب از معادله نشان داده شده در شکل استفاده می‌کند.

(۱) توجه: عملیات جمع در ضرب ماتریسی نشان داده شده در شکل ۵ در مبنای ۲ (عملیات XOR) می‌باشد.

۴ پیاده‌سازی طرح رمز COCO-128 در مد EBC



شکل ۶ شمای مد EBC در رمزهای قالبی

قرار است به وسیله این پیاده‌سازی بتوانیم یک متن را رمز کنیم. بنابراین ابتدا لازم است که پیام خود را کدگذاری کنیم. روش کدگذاری پیام به خود شما واگذار شده است. سپس به منظور رمزگذاری، همانطور که در شکل ۶ نشان داده شده پیام کدگذاری شده خود را به بلوک‌های ۱۲۸-بیتی تقسیم کرده (اندازه هر قطعه در رمز COCO-128، ۱۲۸-بیت می‌باشد) و هر قطعه را با کلید اصلی K رمز می‌کنیم. به منظور رمزگشایی، متن‌های رمز متناظر با هر بلوک در مرحله رمزگذاری را با کمک کلید اصلی K رمزگشایی می‌کنیم.

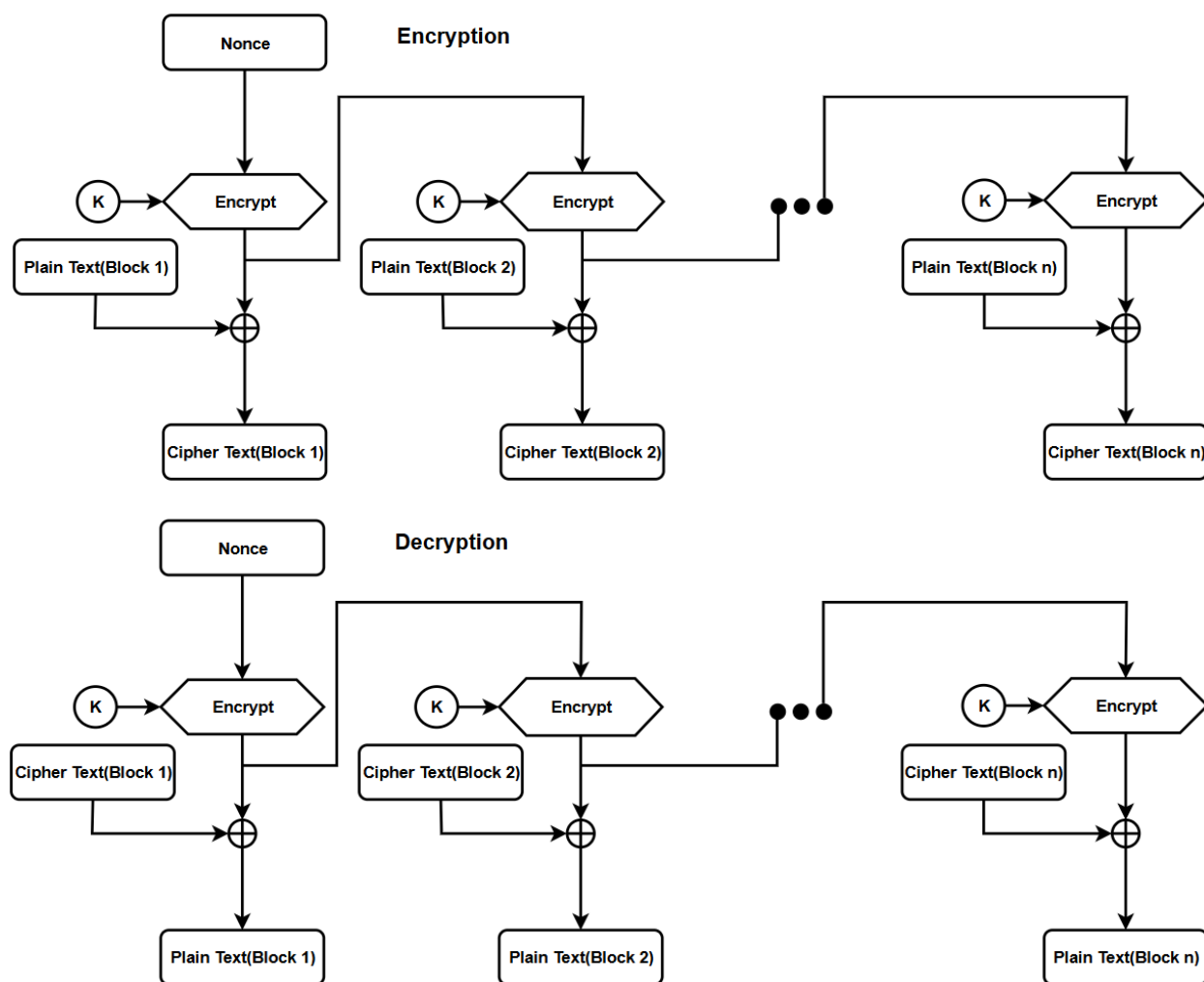
تذکر: در هنگام تقسیم پیام‌ها به بلوک‌های ۱۲۸-بیتی امکان دارد نیاز به padding داشته باشید.

¹ Encode



۵ پیاده‌سازی مُد OFB، CTR و CBC

۵-۱ مُد OFB

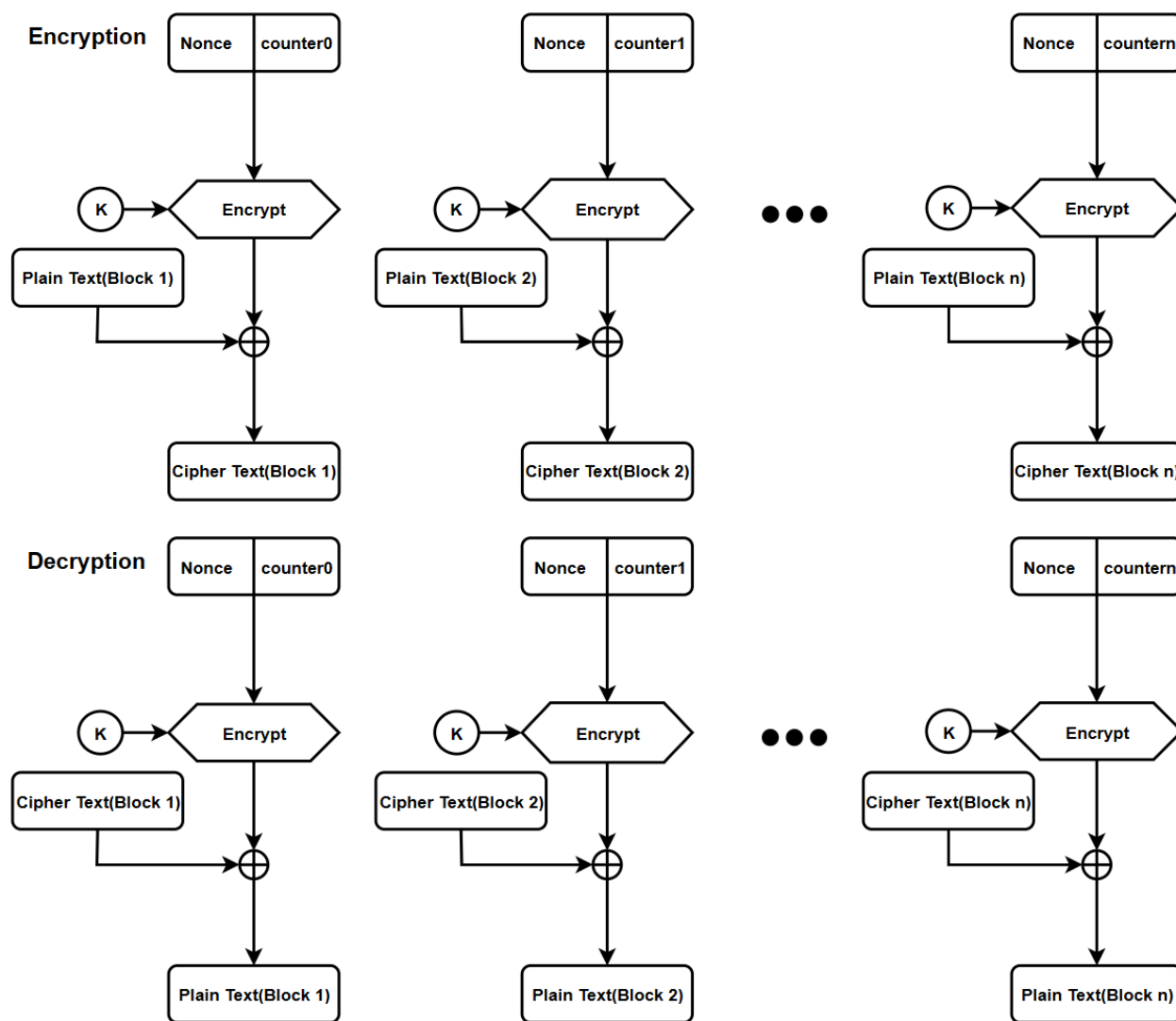


شکل ۷ شمای مُد OFB در رمزهای قالبی

۵-۲ مُد CTR

در شکل ۸ می‌توانید ساختار مُد CTR در رمزهای قالبی را مشاهده کنید. این مُد دارای سه ورودی در هر مرحله می‌باشد؛ (۱) یک مقدار ۸-بایتی **Nonce** (۲) کلید اصلی (۳) بلوک متن معلوم. همچنین شایان توجه است که در این ساختار یک متغیر محلی ۸-بایتی **counter** وجود دارد که مقدار اولیه آن در هنگام رمزگذاری و رمزگشایی باید یکسان باشد. مقدار **Nonce** و **counter** با یک دیگر پیوسته^۱ و یک عبارت ۱۶-بیتی ایجاد می‌کنند که با کلید اصلی توسط الگوریتم رمزنگاری مذکور رمز می‌شوند.

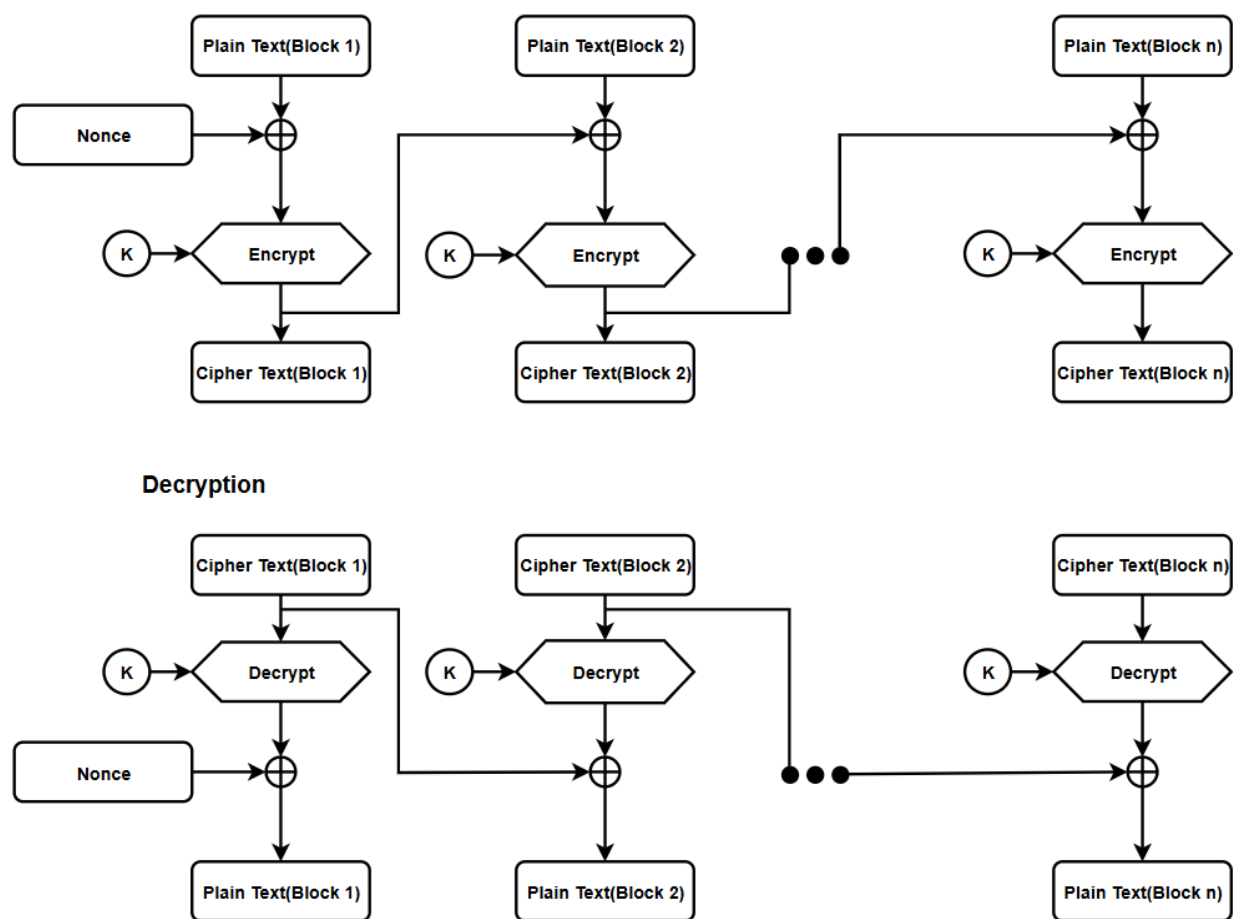
^۱ Concatenate



شکل ۸ شمای مُد CTR در رمزهای قالبی

۳-۵ مد CBC

شکل ۹ ساختار مُد CBC را در رمزهای بلوکی نشان می‌دهد؛ همانطور که در شکل مشاهده می‌شود این مُد دارای سه ورودی می‌باشد (۱) یک مقدار ۱۶-بایتی (Nonce ۲) کلید اصلی (۳) بلوک متن معلوم. در هر مرحله بلوک متن معلوم با مقدار Nonce در پیمانه ۲ جمع و سپس به وسیله کلید اصلی K رمزگذاری می‌شود. نکته شایان توجه در این ساختار استفاده از ماژول Decrypt در هنگام رمزگشایی می‌باشد؛ که با دو مُد پیشین متفاوت است (در مُد OFB و CTR در هنگام رمزگشایی نیز از ماژول Encrypt استفاده می‌شود).



شکل ۹ شمای مُد CBC در رمزهای قالبی

